

SPSC 无锁环形缓冲区

内存序、缓存索引优化与无符号溢出正确性的第一性原理论述

Robert (Bo) Hou · Robert's Technical Notes · 2026 年 6 月

摘要。单生产者单消费者(SPSC)无锁环形缓冲区是低延迟系统的核心数据骨架。本报告从第一性原理出发,分三个层次展开设计。第一,推导每个原子操作所需的最小内存序,并指出一个关键区分:单调性论证只确立值安全,而非 C++ 所要求的 happens-before 关系——因此槽位复用迫使容量游标加载必须使用 acquire。第二,分析缓存索引优化,通过单调性论证其安全性,并澄清它降低的是一致性开销,而非正确性或填充速率。第三,给出一个基于同余理论的形式化证明,说明绝对计数设计在无符号硬件溢出下仍然正确,并以不变量维持归纳法加以加固。

1. 基础设计与记号

我们将可平凡复制的 T 类型元素存入大小为 Capacity(2 的幂)的定长数组中。两个单调递增的绝对计数器——tail_(仅生产者写)与 head_(仅消费者写)——通过位掩码索引缓冲区。

```
template <typename T, size_t Capacity>
class SpscQueue {
    static_assert((Capacity & (Capacity - 1)) == 0,
        "Capacity must be power of 2");

    alignas(64) std::atomic<size_t> head_{0}; // consumer writes
    alignas(64) std::atomic<size_t> tail_{0}; // producer writes
    alignas(64) std::array<T, Capacity> buf_;

public:
    bool try_push(const T& item) { // producer only
        size_t t = tail_.load(std::memory_order_relaxed); // [A]
        size_t h = head_.load(std::memory_order_acquire); // [B]
        if (t - h == Capacity) return false; // full
        buf_[t & (Capacity - 1)] = item;
        tail_.store(t + 1, std::memory_order_release); // [C]
        return true;
    }

    bool try_pop(T& out) { // consumer only
        size_t h = head_.load(std::memory_order_relaxed); // [D]
        size_t t = tail_.load(std::memory_order_acquire); // [E]
        if (h == t) return false; // empty
        out = buf_[h & (Capacity - 1)];
        head_.store(h + 1, std::memory_order_release); // [F]
        return true;
    }
};
```

三个结构性决策需要在开篇强调。

- 绝对计数,而非回绕索引。head_ 与 tail_ 永远只增不减;物理槽位是 counter & (Capacity-1)。这使占用量 $t - h$ 成为一次减法,无需任何分支处理回绕。
- 2 的幂容量。掩码 & (Capacity-1) 恰好等于 mod Capacity,消除了一次除法。
- 按缓存行对齐。head_ 与 tail_ 占据不同的缓存行(aligned(64))以避免伪共享:生产者只写 tail_,消费者只写 head_;若把二者放在一起,即便它们逻辑上互相独立,该缓存行也会在两核间因 MESI 协议而反复弹跳(ping-pong)。

2. 内存序:最小规格

2.1 两个独立的事实:原子性 vs. 顺序性

必须区分 `std::atomic` 提供的两类保证。

- 原子性:原子加载/存储不可分割(不会读到撕裂值),且对任何单个原子变量,所有线程都观察到一致的修改顺序(modification order)——即便在 relaxed 下也是如此。
- 顺序性:针对不同变量的操作,是否以某种特定顺序对另一线程可见。这才是 memory_order 所控制的东西。

relaxed 操作给你前者,但不给后者。这正是为何读取自己的计数器用 relaxed 永远安全:凭借“读到自己最新写入(read-your-own-write)”加上单变量修改顺序的一致性,生产者对 tail_ 的 relaxed 加载 [A] 与消费者对 head_ 的 relaxed 加载 [D],总能观察到各自的最新存储。

2.2 release/acquire 握手

- release(发布方):在宣告“就绪”的那个存储上,所有序在其前的写入,都保证对任何随后读到该宣告值的线程可见。它是一道单向的“向下”栅栏——前面的写入不能下沉到它之后。
- acquire(读取方):在读到该宣告值的那个加载上,所有序在其后的读取都不能被提到它之前。它是一道单向的“向上”栅栏。

握手仅当 acquire 加载真正读到了 release 存储所写的值时才触发。在那一刻,一条 synchronizes-with 边被建立,发布方在其 release 之前写的一切,都对读取方在其 acquire 之后变得可见。

注记。 release/acquire 不必“对称”。一个 release 存储与一个 relaxed 加载配对仍然合法;那个 relaxed 加载只是不享受同步而已。一个加载是否需要 acquire,取决于该线程在加载之后接下来做什么,而非取决于对面的存储是不是 release。

2.3 容易的加载:[E]

[E] — 消费者加载 tail_:必须 acquire。加载之后,消费者执行 `out = buf[h]`,读取的是生产者在其 release 存储 [C] 之前通过 `buf[t] = item` 填入的槽位。关键在于:地址 `buf[h]` 依赖的是 `h`(消费者自己的计数器),而非加载得到的 `t`。因此并没有数据依赖强制“读载荷”等待“加载 tail_”;二者之间只隔着一道经由 `if (h == t)` 的控制依赖,而控制依赖并不能阻止投机执行。CPU 可能在加载尚未完成前就投机地执行 `out = buf[h]`,读到陈旧或写了一半的数据。[E] 上的 acquire 禁止这种重排;配合 [C] 的 release,它保证一旦观察到新的 `t`,`buf[h]` 必已被完整写入。到此判据仍然直观:加载者随后要读另一线程发布的载荷,故必须同步。

2.4 困难的加载 [B]:三个层次的理解

位置 [B]——生产者对 `head_` 的加载——是真正精彩的推理所在,而最诚实的讲法,是把它呈现为一次由浅入深、逐级攀升的理解过程。每一层手里掌握的知识不同,于是每一层“获胜”的答案也不同。结论是:答案沿着 `acquire` → `relaxed` → `acquire` 走了一圈——你一开始被迫选 `acquire`,一个聪明的洞察把你诱到 `relaxed`,而 C++ 语言标准又把你逼回 `acquire`。把这条弧线走一遍——而不只是落到最终答案——正是“真正理解机器”与“背诵一条规则”的分水岭。

第一层——还不知道单调性这个窍门:`acquire` 获胜

假设你只从代码结构出发,没有任何特殊洞察。你盯着 `push` 路径,试图用依赖关系来论证用最便宜的内存序 `relaxed`:

```
size_t h = head_.load(/* ? */); // [B]
if (t - h == Capacity) return false;
buf_[t & (Capacity - 1)] = item; // [][]
```

诱人的论证是:“第 2 行用了 `h`,第 3 行被第 2 行守卫,所以这次写被串在加载 `head_` 之后,不会被重排到它前面——因此 `relaxed` 就够了,硬件会保住这个依赖。”这看起来天衣无缝:现代 CPU 绝不违反真数据依赖(RAW),那何必花钱加栅栏?

这个论证是错的,而看清它为什么错,正是第一层的理解。这次写的地址是 `t & mask`,它依赖 `t`——而非加载得到的 `h`。所以从这次写回到那次加载,没有真数据依赖。把它俩联系起来的,只有 `if (t - h == Capacity)` 这个分支——一条控制依赖。而控制依赖约束不了现代乱序 CPU:它会投机这个分支(赌它没满),提前执行这次写,根本不等加载完成。你以为能锁住顺序的那个依赖并不存在;你倚靠的那个分支,被投机一步跨过了。

于是在第一层,“因为依赖所以用 `relaxed`”这条捷径垮掉了。手里没有别的牌,你无法证明 `relaxed` 安全,保守而正确的选择就是同步:`acquire` 获胜。注意,这已经是正确的答案了——只是还没被证明为唯一的答案。我们还没动用队列的任何巧妙性质。`acquire` 只是一个谨慎的工程师在“没有任何东西授权用更弱内存序”时本能伸手去拿的默认值。

第二层——一旦知道单调性这个窍门:`relaxed` 获胜

现在你引入一个真正巧妙的观察,正是它把“认真琢磨过这个数据结构的人”和“没琢磨过的人”区分开来:`head_` 与 `tail_` 是单调的——它们只增不减(许多工程师从未注意到这一点;它们是绝对计数器,从不递减、从不重置,仅在整数溢出时回绕,而第 4 节会证明那是无害的)。由单调性,可以得到一个干净的安全性证明:

命题(`relaxed` [B] 下的值安全)。由于 `head_` 只增,`relaxed` 加载只可能返回 $\leq$ 真实当前 `head_` 的值。生产者因而低估可用空间。最坏的情况是假满——生产者在其实有空位时拒绝写入——这无害,且在下次调用即自我纠正。危险的反面,即假空(在没有空位时误以为有,从而覆盖活数据),不可能发生。故生产者绝不算出越界索引,也绝不在逻辑上覆盖未读槽位。

这个证明是对的,而且很漂亮。在纯算法、状态机的意义上,单调性恰好把第一层缺失的依赖所未能提供的安全性买了回来。如果你是在硬件上实现这个 FIFO——比如 FPGA 上的 Verilog FIFO,唯一的问题只是逻辑是否自洽——你完全可以名正言顺地去掉栅栏、省下门电路。第二层落幕时,`relaxed` 看起来不只可接受,而且最优:可证安全,还省下一次同步。`relaxed`

获胜。正是在这一层,一个很强的工程师,在对算法的推理完全正确的情况下,自信地写下了 relaxed——然后即将被语言伏击。

第三层——一旦懂得 C++ 标准:acquire 再次获胜

伏击在于:C++ 评判的不是你的算法。编译器依据 C++ 内存模型裁决,而它对数据竞争的定义是纯形式化的,对你的聪明无动于衷:

C++ 数据竞争(非正式)。若两个线程访问同一个非原子对象,至少一个是写,且二者之间没有 happens-before 关系,则程序行为未定义——与物理时序无关。

要看清这个陷阱,追踪单个槽位 buf_[i] 在环绕一圈、槽位被复用时的情形。四个事件横跨两个线程:

```
线程 1:      线程 2:
(1) out = buf_[i];      (3) h = head_.load(/* ? */); // [B]
(2) head_.store(release); // [F]      (4) buf_[i] = item; // []
```

事件 (1) 与事件 (4) 触碰的是同一个非原子

buf_[i]:消费者读它,生产者随后覆盖它。其中一个写。要无竞争,读 (1) 必须 happens-before 覆盖 (4)。在每个线程内部,顺序是天然的:(1) 序前于 (2),(3) 序前于 (4)。缺的是 (2) 与 (3) 之间那条跨线程的边——而这条边仅当 [B] 以 acquire 读到 [F] 的 release 值、形成 synchronizes-with 时才存在。

这正是 [F] release $\leftrightarrow$ [B] acquire

的配对。它是一条反向同步边——方向从消费者指向生产者:“消费者已读完 buf_[i] happens-before 生产者方可覆盖它。”它是正向边 [C] release $\leftrightarrow$ [E] acquire

的镜像,后者方向从生产者指向消费者(“生产者已写完 buf_[t] happens-before 消费者方可读它”)。两条边方向相反、连接不同的操作对;合在一起,它们让每个槽位在两个线程之间安全轮转。两条边都不是装饰:正向边阻止消费者读到生产者还没写完的数据;反向边阻止生产者覆盖消费者还没读完的数据。

现在是判决。若 [B] 为 relaxed,生产者仍在数值上观察到新的 head_(relaxed 遵守每个变量的 modification order),故在物理时间上它确实等待了——第二层的单调性证明并没有在硬件层面说谎。但 C++ 抽象机在 (2) 与 (3) 之间不记录任何 synchronizes-with 边。没有边,(1) 就不 happens-before (4);两个线程对非原子 buf_[i] 一读一写且无定序——数据竞争——程序即未定义行为。一旦 UB,编译器便可在 -O3 下自由地重排、合并、乃至彻底删除对 buf_ 的访问;第二层苦心证明的物理时序安全就此灰飞烟灭,因为标准从未承诺会兑现它。acquire 再次获胜——这一次是硬性要求,而非默认。

2.4.1 和解:两个定理,一句口号

这三层并不矛盾;它们回答的是逐级更微妙的问题。第一层确立了依赖直觉无法授权 relaxed(控制依赖 $\neq$ 数据依赖,且投机跨过了分支)。第二层用单调性确立了值安全——算法只算出正确的索引。第三层确立了 C++ 的合法性关乎 happens-before 形式,而非值安全,且槽位复用因此需要那条反向的 [F] $\rightarrow$ [B] 边。决定性的领悟是:关于值安全的证明,并非关于无数据竞争的证明:它们是关于不同事物的两个不同定理,而无锁代码必须同时赢下二者。单调性证明了“值是对的”,却无法凭空制造出抽象机所要求的那条 happens-before 边。

所以 [B] 的口号是:acquire 是诚实的默认;单调性诱使你降级到 relaxed,甚至在“值”的意义上证明了它安全;但标准拒绝,因为值安全不等于无数据竞争——于是 acquire 不动。这正是 Folly 的 `ProducerConsumerQueue`、`Boost.Lockfree` 以及一切工业级 SPSC 在此处采用 acquire 的原因。那段绕道 relaxed 的弯路不是需要藏起来的难堪;把它走一遍,恰恰证明你同时掌握了算法与 C++ 内存模型,而不是背诵一句“总是用 acquire”。

2.5 六个内存序汇总

位置	操作	内存序	理由
[A]	<code>tail_.load(□□)</code>	relaxed	读到自己最新写入
[B]	<code>head_.load(□□)</code>	acquire	为槽位复用锚定反向同步
[C]	<code>tail_.store</code>	release	发布 buf_ 载荷
[D]	<code>head_.load(□□)</code>	relaxed	读到自己最新写入
[E]	<code>tail_.load(□□)</code>	acquire	之后要读载荷 buf_[h]
[F]	<code>head_.store</code>	release	读完成后才释放槽位

承重的握手有两条,每条游标各一。tail 链 [C] release ↔ [E] acquire 承载载荷的正向可见性(生产者写 `buf_[t]` → 消费者读 `buf_[h]`)。head 链 [F] release ↔ [B] acquire 承载使槽位复用安全的反向同步(消费者读完 `buf_[i]` → 生产者方可覆盖)。去掉任一链的 acquire/release,都会切断 buf_ 在某一方向的 happens-before,从而引入数据竞争。

2.6 平台落地

在 x86(TSO)上,relaxed/acquire/release 这一整族加载与存储都编译为普通的 mov;硬件本就禁止除 store→load 之外的所有重排。只有 seq_cst 存储会带来额外开销(一条 xchg 或 mov+mfence 以排空 store buffer),这正是低延迟代码避开 seq_cst 的原因。在弱序架构(ARM、POWER)上,acquire/release 会发出真正的屏障指令,[B]、[C]、[E]、[F] 四者届时都下降为真实栅栏,任何一个都不能降为 relaxed,否则会切断一条 happens-before 链。该抽象是可移植的:工程师只需就“数据 + 旗子”的happens-before 进行推理,编译器会把它下降为各平台正确的指令。

3. 缓存索引优化

3.1 动机:一致性是正确的,但不是免费的

每一次加载 `head_` 的 `try_push` 都要付出跨核成本。消费者最近写过 `head_`,故其缓存行在消费者核上处于 `Modified` 状态。生产者读它时,MESI 必须跨核转移所有权——几十到上百个周期——而随后消费者的写又会夺回该行,造成 ping-pong。MESI 保证生产者读到最新值,但读这个动作本身很贵。该优化消除了其中大部分跨核读。

3.2 机制

生产者持有一个私有的、非原子的 `cached_head_`。大多数 `push` 用这个缓存值判断是否已满,只触碰生产者自己的 L1。只有当缓存值显示“已满”时,生产者才执行一次真正的 `head_.load()` 去刷新。

```
bool try_push(const T& item) {
    size_t t = tail_.load(std::memory_order_relaxed);
    if (t - cached_head_ == Capacity) {           // cheap local test
        cached_head_ = head_.load(std::memory_order_acquire); // refresh
        if (t - cached_head_ == Capacity)
            return false;                         // genuinely full
    }
    buf_[t & (Capacity - 1)] = item;
    tail_.store(t + 1, std::memory_order_release);
    return true;
}
```

3.3 安全性

引理(缓存单调性)。在任何时刻, $\text{cached_head_} \leq \text{actual_head}$,其中 `actual_head` 是 `head_` 的当前值。

证明。`cached_head_` 只可能被某次对 `head_` 的过往加载所赋值。由于消费者单调地递增 `head_`,任何先前观察到的值都 $\leq$ 当前值。 $\square$

命题(保守正确性)。若生产者写入某个槽位,则该槽位确实空闲;生产者绝不覆盖未读数据。

证明。生产者仅当其满判断失败时才写入,即(可能刷新之后) $t - \text{cached_head_} < \text{Capacity}$ 。由单调性引理, $t - \text{actual_head} \leq t - \text{cached_head_} < \text{Capacity}$ 。因此真实占用严格低于容量,故存在空闲槽位。缓存值只可能造成虚假已满(一次保守的拒绝),绝不会造成虚假空闲;“假满”无害(它触发一次刷新),而会导致数据损坏的“假空”不可能发生。 $\square$

3.4 它不做什么

该优化并不节流或控制生产者的填充速率,也不支配缓冲区填满的快慢。填充速率完全由生产者与消费者的相对吞吐决定。缓存索引改变的只是满判断这一检查的成本——用偶尔一次的跨核加载替代频繁的跨核加载。生产者最终受限于物理容量边界,而非缓存。打个比方:缓存索引是一把更便宜的量尺,用来读取水位;换量尺并不改变水流入或流出的快慢。

4. 无符号硬件溢出下的正确性

本节形式化地说明:当 k 位计数器溢出回绕时,绝对计数设计为何仍然正确。我们先确立算法所维持的不变量,再在该不变量下证明寻址不变性与距离不变性。

4.1 形式化设定

设 $\mathbb{Z}_{2^k}$ 为模 2^k 的整数环(一个 k 位无符号寄存器,通常 $k = 64$)。无符号加减定义为 $a \oplus b \equiv a + b \pmod{2^k}$ 与 $a \ominus b \equiv a - b \pmod{2^k}$ 。设 $N = 2^m$, 其中 $0 < m < k$ 为容量,寻址映射 $f: \mathbb{Z}_{2^k} \rightarrow \mathbb{Z}_N$ 为位截断 $f(x) = x \text{ AND } (2^m - 1)$ 。

引理(f 的模同态性)。由于掩码 $(2^m - 1)$ 隔离出低 m 位,故 $f(x) \equiv x \pmod{2^m}$ 。又因 $2^m \mid 2^k$, 该映射可降到商环并保持加法结构: $f(a \oplus b) \equiv (a + b \pmod{2^k}) \pmod{2^m} \equiv (a + b) \pmod{2^m} \equiv (f(a) + f(b)) \pmod{2^m}$ 。

4.2 所维持的不变量

设 $t, h \in \mathbb{Z}_{2^k}$ 为生产者与消费者的绝对计数。

不变量(无溢出占用)。在每一个可达状态下, $0 \leq (t \ominus h) < N$ 。

后续所有正确性都依赖这个不变量,因此我们证明算法维持它,而不是假设它。

定理 1(不变量维持)。在单生产者单消费者运行下,不变量 $0 \leq t \ominus h < N$ 在每个可达状态下均成立。

证明。对操作做归纳。

基例。初始时 $t = h = 0$, 故 $t \ominus h = 0$, 且 $0 \leq 0 < N$ 。

归纳步。假设 $0 \leq t \ominus h < N$ 。两种操作可改变状态。

Push。生产者仅在其守卫验证 $t \ominus h < N$ (即 $t \ominus h \leq N - 1$) 之后才推进 $t \mapsto t \oplus 1$ 。递增后占用量变为 $(t \oplus 1) \ominus h = (t \ominus h) + 1 \leq (N - 1) + 1 = N$ 。结合下界,新占用量落在 $[1, N]$ 内,处于可接受范围。(在极值 N 处缓冲区恰好满,下一次 push 守卫会正确拒绝。)只有生产者修改 t , 故 h 不变,下界 $0 \leq t \ominus h$ 得以保持。

Pop。消费者仅在验证 $t \ominus h > 0$ (非空)之后才推进 $h \mapsto h \oplus 1$ 。递增后 $t \ominus (h \oplus 1) = (t \ominus h) - 1 \geq 0$, 下界得以保持;占用量严格减小,故上界亦保持。只有消费者修改 h , 故 t 不变。

由于生产者只改 t 、消费者只改

h , 两个更新从不在同一计数器上竞争,且各自都保持两条边界。因此该不变量在所有可达状态下成立。□

4.3 寻址不变性

定理 2(溢出处寻址无缝)。当生产者计数 $t_1 = 2^k - 1$ 递增到 $t_2 = t_1 \oplus 1 = 0$ 时,物理索引仍保持连续: $f(t_2) = (f(t_1) + 1) \pmod{N}$ 。

证明。左端。 $t_2 = 0$, 故 $f(t_2) = f(0) \equiv 0 \pmod{2^m}$, 又因 $0 \in [0, 2^m - 1]$, 得 $f(t_2) = 0$ 。

右端。 $(f(t_1) + 1) \pmod{2^m} \equiv (t_1 + 1) \pmod{2^m} \equiv (2^k - 1 + 1) \pmod{2^m} \equiv 2^k \pmod{2^m}$ 。因 $2^m \mid 2^k$, 此值为 0。

两端皆等于 0,由相等的传递性得 $f(t_2) = (f(t_1) + 1) \bmod N = 0$ 。绝对计数的回绕在物理槽位序列上不产生任何不连续。 $\square$

4.4 距离不变性

定理 3(经环算术的距离不变性)。 设生产者已回绕,使其名义值低于消费者: $t_2 = 0 < h$,其中 $h = 2^k - d$, d 为真实物理距离, $0 < d < N$ 。则无符号减法无需分支即得精确距离: $t_2 \ominus h = d$ 。

证明。在 $\mathbb{Z}_{2^k}$ 中,减法即加上加法逆元: $t_2 \ominus h \equiv (t_2 + 2^k - h) \bmod 2^k = (0 + 2^k - (2^k - d)) \bmod 2^k = d \bmod 2^k$ 。由所维持的不变量, $0 < d < N < 2^k$,故 d 落在基本域内,外层约化为恒等: $d \bmod 2^k = d$ 。于是 $t_2 \ominus h = d$ 。 $\square$

注记(补码是实现,而非定义)。 因 t 、 h 为无符号,实际起作用的代数是模 2^k 的环算术。项 $2^k - h$ 是 h 在此环中的加法逆元;其位模式恰好与补码编码($\text{NOT } h + 1$)一致,这正是为何 ALU 上一次无符号减法即便在数值上 $t < h$ 时也能给出正确距离。硬件免费地纠正了这种表面上的“时间倒置”。

4.5 位流演算示例

设 $k = 8(2^k = 256)$ 与 $N = 8$ 。设 $h = 250(1111\ 1010)$,生产者回绕到 $t = 0(0000\ 0000)$;真实距离为 $256 - 250 = 6$ 。

- h 的加法逆元: $\text{NOT } 1111\ 1010 + 1 = 0000\ 0101 + 1 = 0000\ 0110$ (值 6)。
- 无符号加 $t + (2^k - h)$: $0000\ 0000 + 0000\ 0110 = 0000\ 0110$ (值 6)。
- 掩码 $\& (N-1) = \& 0000\ 0111$: $0000\ 0110 \text{ AND } 0000\ 0111 = 0000\ 0110$ (槽位 6)。

模算术的循环拓扑以零分支、零误预测惩罚吸收了这次回绕。

5. 综述

SPSC 环形缓冲区奖励在恰当抽象层级上的推理——也惩罚把两种“正确性”混为一谈。值安全(生产者绝不算出坏索引)源自游标单调性;但 C++ 的合法性是 happens-before 形式问题:每一对对 `buf_` 的并发非原子访问都必须由一条 `synchronizes-with` 边定序,否则无论物理时序如何,程序都存在数据竞争。这迫使两条游标链都必须用 `acquire/release`——正向(载荷发布)与反向(槽位复用)——这正是一切工业实现的共同做法。相比之下,寻址的正确性归结为两条模运算事实:掩码是一个同态,无符号减法就是环减法。底层硬件机制——`store buffer`、缓存一致性、投机执行——解释了这些抽象为何成立,但实际工作的工程师所推理的,是 happens-before 边、单调计数器与同余;而最关键的一点是:一个关于值安全的算法证明,并不等于一个关于无数据竞争的证明。代码正是在这一区分的层级上被写就,也在这一层级上被辩护。